PYTHON PROGRAMMING

CASE STUDIES

Activity-Based Learning Portfolio

10 Practical Python Problems

- Scholarship allocation and electricity billing
- Hospital billing and employee payroll
- Telecommunications and railway fare calculations
- Retail billing and manufacturing cost analysis
- Travel budget and campus event planning

Problem statements • Algorithms • Flowcharts • Python code • Test cases

1. University Scholarship Allocation

Problem statement

Build a program that checks scholarship applicants against aggregate marks, minimum marks in every required subject, attendance, and family-income limits. Eligible applicants are ranked, with ties resolved by predefined rules.

Objectives

• Validate applicant data and eligibility criteria.
• Compute aggregate percentage and apply all minimum thresholds.
• Rank eligible applicants using marks, attendance, then lower family income as tie-breakers.
• Display eligibility, rank, and reasons for rejection.

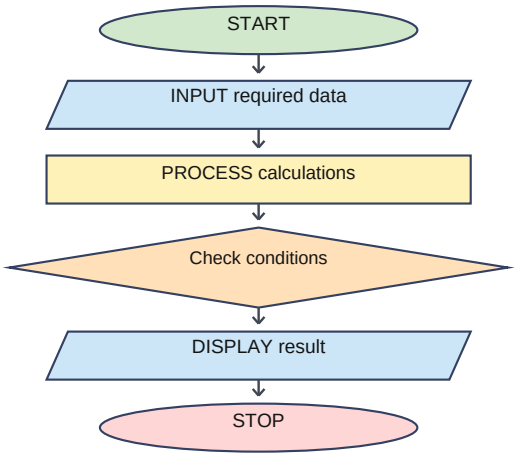
Assumptions

• Marks are percentages from 0 to 100; subject marks are entered as a list.
• Eligibility requires aggregate >= 80, every subject >= 60, attendance >= 75%, and annual family income <= 300000.
• Tie-break order: higher aggregate, higher attendance, lower income, then earlier input order.
• The sample uses three subjects and a fixed scholarship quota of two; thresholds can be changed.

Algorithm

1. Read applicant count and each applicant's name, three subject marks, attendance, and annual family income.
2. Calculate aggregate as the mean of the three subject marks.
3. Mark an applicant eligible only if all four rules pass.
4. Store eligible records as tuples: (aggregate, attendance, negative income, input order, name).
5. Sort eligible tuples in descending order; input order is stored as negative order so earlier entries win exact ties.
6. Assign ranks and display up to the quota as scholarship recipients.

Flowchart



Pseudocode

```
START
INPUT applicant_count
FOR each applicant
    INPUT name, marks, attendance, income
    aggregate = sum(marks) / number_of_subjects
    eligible = aggregate >= 80 AND all marks >= 60
               AND attendance >= 75 AND income <= 300000
    IF eligible
        STORE (aggregate, attendance, -income, -input_order, name)
    ELSE
        DISPLAY not eligible
SORT stored records in descending order
DISPLAY ranked list and scholarship quota
STOP
```

Implementation and testing

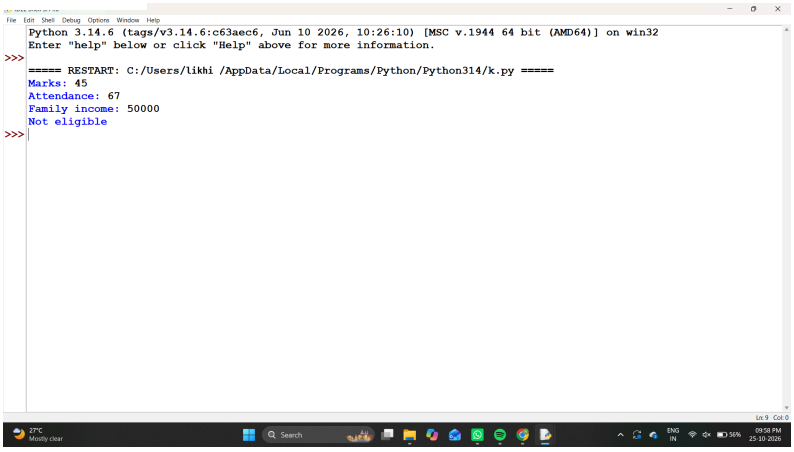
Python source code — exactly four lines

```
n=int(input("Applicants: ")); a=[]
for i in range(n):
 name=input("Name: "); m=[float(input("Subject mark: ")) for _ in range(3)]; att=float(input("Attendance: ")); inc=float(inpu
a.sort(reverse=True); [print(i+1,x[4],round(x[0],2),"AWARDED" if i<2 else "") for i,x in enumerate(a)]
```

Test cases

Input / scenario	Expected result
Asha: 90, 85, 88; attendance 92; income 200000	Eligible; aggregate 87.67; ranked by score.
Bala: 90, 55, 95; attendance 95; income 150000	Not eligible: one subject below 60.
Chitra: 80, 80, 80; attendance 80; income 350000	Not eligible: income exceeds limit.
Dev: 88, 88, 88; attendance 90; income 250000	Eligible; lower rank than equal-score applicant with lower income.

Sample output screen



Conclusion

This program demonstrates eligibility screening, tuple-based ranking, and deterministic tie-breaking using the stated scholarship policy.

2. Progressive Electricity Billing

Problem statement

Calculate monthly household electricity bills using progressive consumption slabs, fixed charges, surcharges, and rebates. Compare several monthly bills and flag unusual increases.

Objectives

• Apply progressive slab rates without charging the highest rate to all units.
• Add fixed charges and surcharge, then apply a qualifying rebate.
• Compare bills over consecutive months and flag increases above a defined threshold.

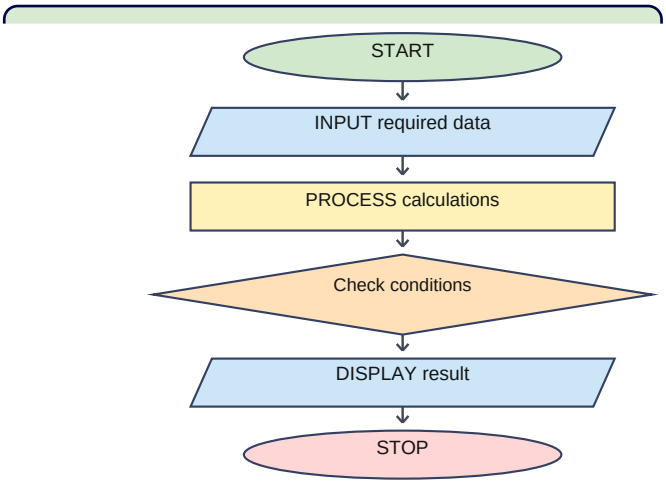
Assumptions

• Slabs: first 100 units at ■2/unit, next 100 at ■3/unit, units above 200 at ■5/unit.
• Fixed charge ■75; surcharge is 5% of energy charge; rebate ■50 if consumption is at most 100 units.
• An increase is unusual when the bill rises by more than 30% from the previous month.
• Consumption must be a non-negative number.

Algorithm

1. Read number of months and each month's units.
2. Calculate energy charge slab by slab.
3. Add fixed charge and 5% surcharge; subtract rebate if eligible.
4. Store monthly bill in a list and display it.
5. From month two onward, calculate percentage increase over previous bill and flag increases above 30%.

Flowchart



Pseudocode

```
START
INPUT month_count
FOR each month
    INPUT units
    IF units <= 100: energy = units * 2
    ELSE IF units <= 200: energy = 100*2 + (units-100)*3
    ELSE: energy = 100*2 + 100*3 + (units-200)*5
    bill = energy + 75 + energy*0.05
    IF units <= 100: bill = bill - 50
    STORE bill
    DISPLAY bill
    IF not first month AND increase_percent > 30
        DISPLAY UNUSUAL INCREASE
STOP
```

Implementation and testing

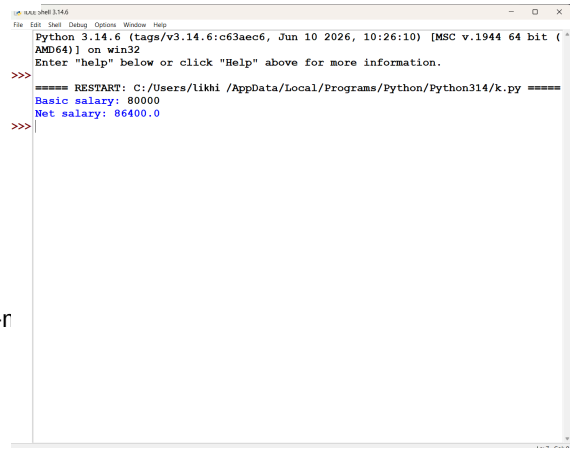
Python source code — exactly four lines

```
n=int(input("Months: ")); bills=[]
for i in range(n):
 u=float(input("Units: ")); e=2*min(u,100)+3*min(max(u-100,0),100)+5*max(u-200,0); b=e+75+e*.05-(50 if u<=100 else 0); bills.
 if len(bills)>1 and bills[-2]>0 and (b-bills[-2])/bills[-2]>.30: print("UNUSUAL INCREASE")
```

Test cases

Input / scenario	Expected result
80 units	Energy ■160; surcharge ■8; fixed ■75; rebate ■50; bill ■193.
150 units	Energy ■350; surcharge ■17.50; fixed ■75; bill ■442.50.
Month bills ■193 then ■442.50	Increase 129.3%; flagged unusual.
-5 units	Invalid consumption.

Sample output screen



Conclusion

The slab calculation and month-to-r and lists can identify bill changes.

3. Hospital Billing and Patient Classification

Problem statement

Prepare patient bills from consultation, diagnostic, and treatment charges; apply concessions, validate constraints, and classify the patient's payment status according to a supplied policy.

Objectives

• Calculate itemized and total charges.
• Apply concession rules consistently.
• Reject negative charges and invalid payments.
• Classify accounts as PAID, PARTIALLY PAID, or OUTSTANDING.

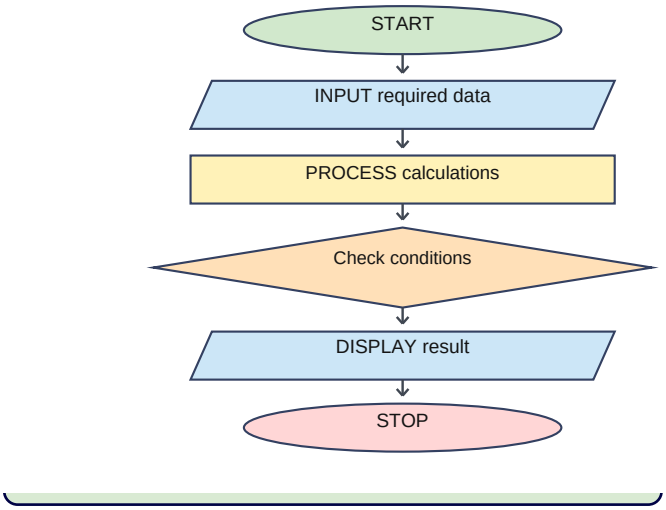
Assumptions

• All charges are non-negative; concession is 10% when total charges are at least ■5000.
• Payment must be between ■0 and final bill inclusive.
• Balance zero means PAID; payment greater than zero but less than bill means PARTIALLY PAID; zero payment means OUTSTANDING.
• No insurance or tax is modeled.

Algorithm

1. Read patient name and consultation, diagnostic, treatment charges.
2. Validate that all charges are non-negative.
3. Compute gross bill and concession, then final bill.
4. Read payment and validate it is between zero and final bill.
5. Calculate balance and classify account.
6. Print an itemized statement.

Flowchart



Pseudocode

```
START
INPUT name, consultation, diagnostics, treatment
IF any charge < 0: DISPLAY INVALID; STOP
gross = consultation + diagnostics + treatment
IF gross >= 5000: concession = gross * 0.10
ELSE: concession = 0
final_bill = gross - concession
INPUT payment
IF payment < 0 OR payment > final_bill: DISPLAY INVALID PAYMENT
ELSE
    balance = final_bill - payment
    IF balance == 0: status = PAID
    ELSE IF payment > 0: status = PARTIALLY PAID
    ELSE: status = OUTSTANDING
    DISPLAY invoice
STOP
```

Implementation and testing

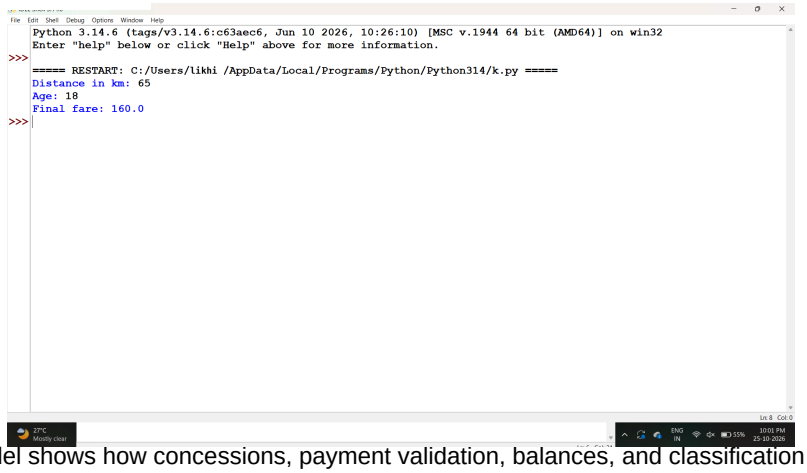
Python source code — exactly four lines

```
name=input("Patient: "); c=float(input("Consultation: ")); d=float(input("Diagnostics: ")); t=float(input("Treatment: "))
g=c+d+t; disc=g*.10 if g>=5000 else 0; bill=g-disc; p=float(input("Payment: "))
if min(c,d,t,p)<0 or p>bill: print("Invalid charges/payment")
else: print(name,"Gross",g,"Concession",disc,"Bill",bill,"Balance",bill-p,"Status",("PAID" if p==bill else "PARTIALLY PAID" i
```

Test cases

Input / scenario	Expected result
■1000 + ■2000 + ■3000; payment ■2000	Gross ■6000; concession ■600; final ■5400; balance ■3400; PARTIALLY PAID.
Charges ■100 + ■200 + ■300; payment 0	Final ■600; OUTSTANDING.
Any charge -■5	Invalid charge.
Final bill ■600; payment ■700	Invalid payment.

Sample output screen



Conclusion

The itemized billing model shows how concessions, payment validation, balances, and classification can be implemented with conditionals.

4. Employee Payroll and Deduction Analysis

Problem statement

Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions and print a formatted salary statement.

Objectives

• Compute earnings and gross salary accurately.
• Apply progressive deduction slabs.
• Calculate net salary and provide an itemized statement.

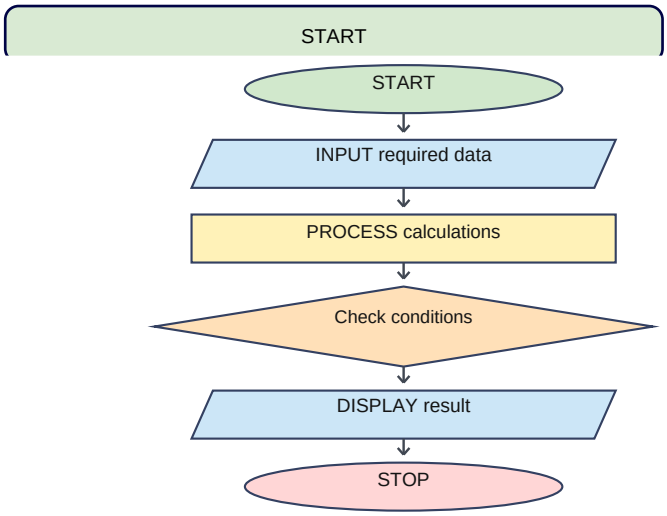
Assumptions

• HRA = 20% of basic; other allowance is entered by user; overtime pay is hours × ■200; incentive is entered.
• Deduction on gross: 0% for first ■15000, 5% on next ■15000, 10% on amount above ■30000.
• All input amounts and overtime hours must be non-negative; this is an educational policy, not tax advice.

Algorithm

1. Read employee name, basic pay, other allowance, overtime hours, and incentive.
2. Calculate HRA, overtime pay, and gross salary.
3. Calculate deduction progressively across the three bands.
4. Calculate net salary as gross minus deduction.
5. Display salary statement with rounded amounts.

Flowchart



Pseudocode

```
START
INPUT name, basic, allowance, overtime_hours, incentive
HRA = basic * 0.20
overtime_pay = overtime_hours * 200
gross = basic + HRA + allowance + overtime_pay + incentive
IF gross <= 15000: deduction = 0
ELSE IF gross <= 30000: deduction = (gross-15000)*0.05
ELSE: deduction = 15000*0.05 + (gross-30000)*0.10
net = gross - deduction
DISPLAY earnings, deduction, net
STOP
```

Implementation and testing

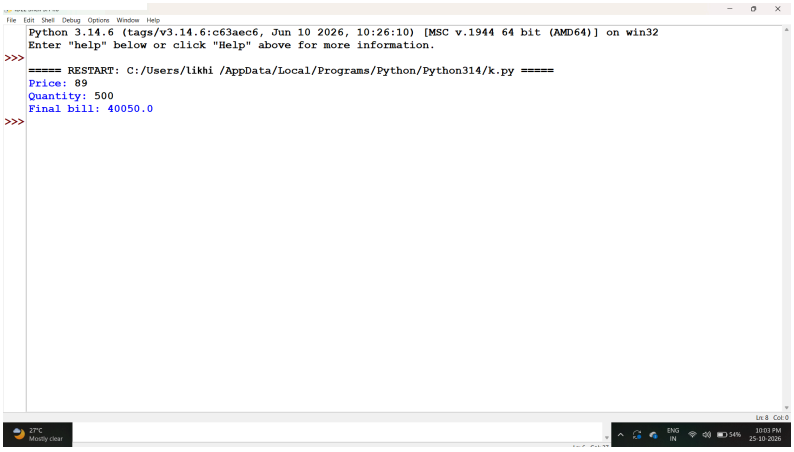
Python source code — exactly four lines

```
name=input("Employee: "); b=float(input("Basic: ")); a=float(input("Allowance: ")); h=float(input("Overtime hours: ")); i=flo
g=b*1.2+a+h*200+i; d=0 if g<=15000 else (g-15000)*.05 if g<=30000 else 750+(g-30000)*.1
print("Invalid input") if min(b,a,h,i)<0 else print(name,"Gross",g,"Deduction",d,"NET",g-d)
print("Salary statement complete")
```

Test cases

Input / scenario	Expected result
Basic 20000, allowance 2000, overtime 5, incentive 1000	HRA 4000; overtime 1000; gross 28000; deduction ■650; net ■27350.
Gross ■12000	Deduction ■0.
Gross ■40000	Deduction ■750 + ■1000 = ■1750.
Negative basic pay	Invalid input.

Sample output screen



Conclusion

The salary statement demonstrates component-wise earnings, progressive deductions, and net-pay calculation.

5. Multi-Plan Telecommunications Cost Optimization

Problem statement

Compare mobile plans against a customer's data, call, validity, and budget requirements. Calculate overage costs and select the lowest-cost plan that satisfies all constraints.

Objectives

• Evaluate multiple plans using lists and tuples.
• Calculate extra usage charges and total cost.
• Filter plans by validity and budget, then select the most economical qualifying plan.

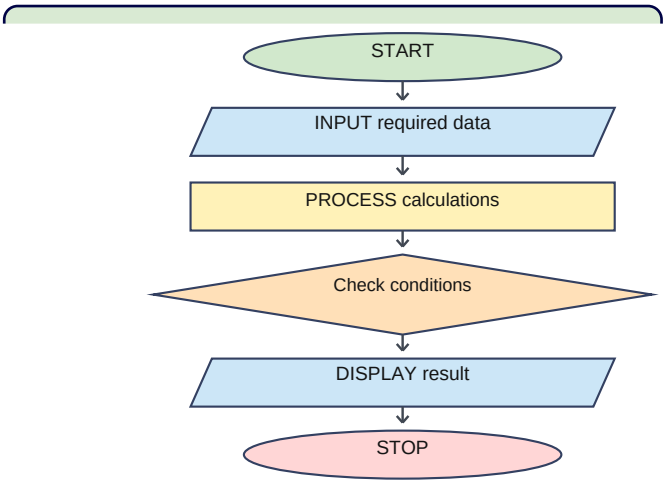
Assumptions

• Each plan is (name, price, included_data_GB, included_call_minutes, validity_days, extra_data_rate, extra_call_rate).
• Extra data costs ■20/GB and extra calls cost ■0.50/minute in the sample plan records.
• A plan is eligible only if validity >= required days and total cost <= budget.
• No prorating or network-quality differences are considered.

Algorithm

1. Read customer data usage, call minutes, required validity, and budget.
2. Read plan records as tuples.
3. For each plan, calculate excess data and minutes, then add overage charges to base price.
4. Keep plans whose validity and total cost satisfy constraints.
5. Track the eligible plan with minimum total cost; display all evaluated plans and the best choice.

Flowchart



Pseudocode

```
START
INPUT data_used, call_minutes, required_days, budget
plans = list of plan tuples
best_cost = very_large_number
FOR each plan
    extra_data = max(0, data_used - included_data)
    extra_calls = max(0, call_minutes - included_calls)
    total = price + extra_data*data_rate + extra_calls*call_rate
    DISPLAY plan and total
    IF validity >= required_days AND total <= budget AND total < best_cost
        best = plan; best_cost = total
IF a best plan exists: DISPLAY best
ELSE: DISPLAY NO SUITABLE PLAN
STOP
```

Implementation and testing

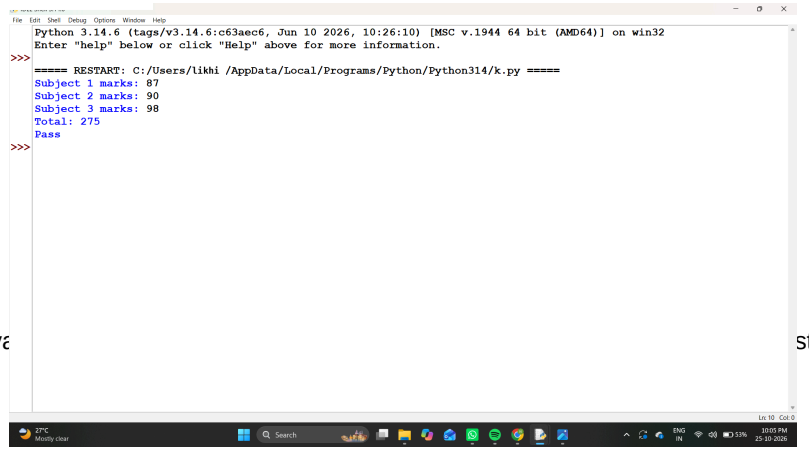
Python source code — exactly four lines

```
p=[("Basic",199,2,200,28,20,.5),("Smart",299,5,500,56,15,.4),("Max",399,10,1000,84,10,.3)]
d=float(input("Data GB: ")); c=int(input("Call minutes: ")); days=int(input("Validity days: ")); budget=float(input("Budget:
costs=[(price+max(0,d-dl)*dr+max(0,c-cl)*cr,name,v) for name,price,dl,cl,v,dr,cr in p]; [print(x[1],round(x[0],2)) for x in c
ok=[x for x in costs if x[2]>=days and x[0]<=budget]; print("Best:",min(ok) if ok else "NO SUITABLE PLAN")
```

Test cases

Input / scenario	Expected result
Usage 4 GB, 300 minutes, 28 days, budget ■350	Basic ■289; Smart ■299; Max ■399; Basic is cheapest eligible.
Usage 8 GB, 800 minutes, 60 days, budget ■500	Max qualifies at ■399; Smart fails validity.
Budget ■100	No suitable plan.
Required validity 90 days	No listed plan qualifies.

Sample output screen



Conclusion

The plan comparison eva stomer's stated requirements.

6. Railway Fare and Reservation Calculator

Problem statement

Calculate fares for several passengers using distance slabs, travel class multipliers, age concessions, reservation fees, and cancellation penalties.

Objectives

• Calculate base fare from distance slabs.
• Apply class factors and age-based concessions.
• Add reservation fees and calculate cancellation refunds/penalties.

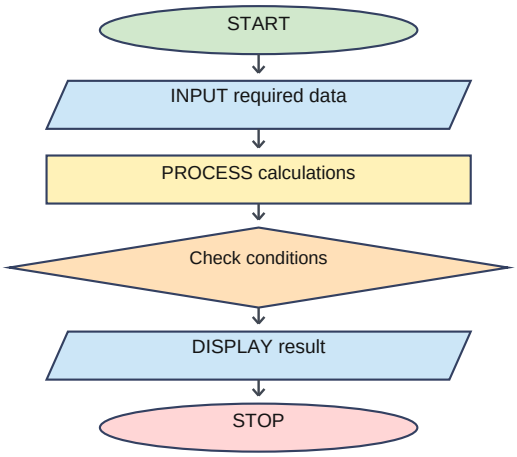
Assumptions

• Base distance rate: ■1.20/km for first 100 km, ■1.00/km for next 400 km, ■0.80/km beyond 500 km.
• Class multipliers: Sleeper 1.0, AC 2.0, Chair 1.4.
• Children under 12 receive 50% fare concession; passengers aged 60+ receive 30%; others receive none. Only one concession applies.
• Reservation fee is ■40 per passenger. Cancellation penalty is 25% of fare before reservation fee; fee is not refunded.

Algorithm

1. Read distance and passenger count.
2. For each passenger, read age and class.
3. Calculate distance-based base fare and multiply by class factor.
4. Apply age concession and add reservation fee.
5. If cancelled, calculate penalty and refund; otherwise show payable fare.
6. Print a fare summary for each passenger.

Flowchart



Pseudocode

```
START
INPUT distance, passenger_count
CALCULATE distance_base using progressive distance slabs
FOR each passenger
    INPUT age, travel_class, cancelled
    fare = distance_base * class_multiplier
    IF age < 12: fare = fare * 0.50
    ELSE IF age >= 60: fare = fare * 0.70
    payable = fare + 40
    IF cancelled:
        penalty = fare * 0.25 + 40
        refund = payable - penalty
        DISPLAY penalty, refund
    ELSE DISPLAY payable
STOP
```

Implementation and testing

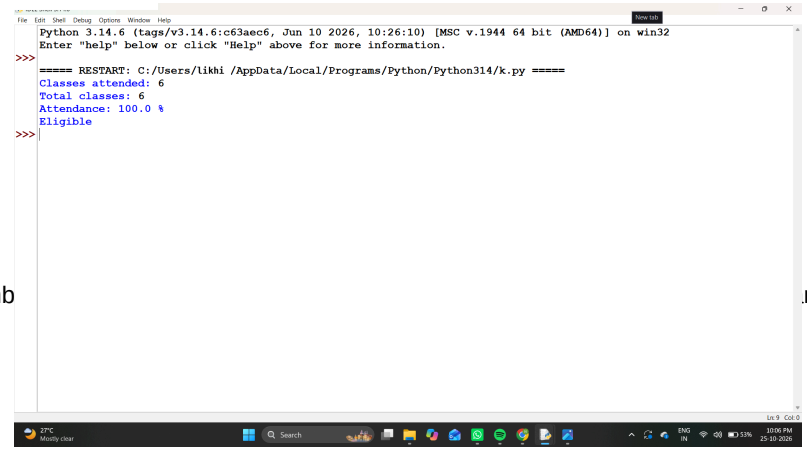
Python source code — exactly four lines

```
dist=float(input("Distance km: ")); n=int(input("Passengers: ")); base=1.2*min(dist,100)+max(0,min(dist-100,400))+max(0,dist-
for _ in range(n):
 age=int(input("Age: ")); cls=input("Class Sleeper/Chair/AC: ").lower(); can=input("Cancelled y/n: ").lower(); f={"sleeper":1
print("Fare calculation complete")
```

Test cases

Input / scenario	Expected result
Distance 200 km; Sleeper adult	Base ■220; payable ■260.
Distance 200 km; AC passenger age 65	Fare ■308 after 30% concession; payable ■348.
Distance 200 km; child in Sleeper	Fare ■110; payable ■150.
Cancellation of ■220 fare	Penalty ■95; refund ■165.

Sample output screen



Conclusion

The fare calculator comb .nd cancellation calculations.

7. Retail Billing with Multiple Discount Rules

Problem statement

Generate an itemized retail invoice for multiple products, applying category-specific taxes, quantity-based prices, threshold discounts, and a maximum concession limit.

Objectives

• Compute line totals and category taxes.
• Apply a bill-level discount when the subtotal reaches a threshold.
• Cap the discount and display the final payable amount.

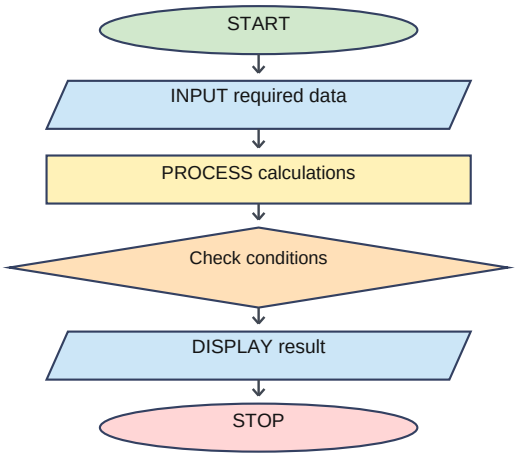
Assumptions

• Categories: grocery tax 5%, electronics tax 18%, clothing tax 12%.
• A 10% discount applies when pre-tax subtotal is at least ■5000; maximum discount is ■1000.
• Product quantity and unit price must be non-negative; category must be valid.
• Discount is applied to subtotal before tax; tax is calculated on discounted subtotal proportionally across the whole bill for this simplified model.

Algorithm

1. Read product count and each product's name, category, unit price, and quantity.
2. Calculate line totals and accumulate subtotal and category tax.
3. If subtotal >= ■5000, calculate 10% discount capped at ■1000.
4. Compute taxable amount as subtotal minus discount and apply weighted tax based on original category line totals.
5. Display each line and the invoice totals.

Flowchart



Pseudocode

```
START
INPUT item_count
subtotal = 0; tax_total = 0
FOR each item
    INPUT name, category, price, quantity
    line_total = price * quantity
    subtotal = subtotal + line_total
    STORE line_total and category tax rate
IF subtotal >= 5000: discount = MIN(subtotal*0.10, 1000)
ELSE discount = 0
FOR each item
    discounted_line = line_total * (subtotal-discount)/subtotal
    tax_total += discounted_line * category_rate
final = subtotal - discount + tax_total
DISPLAY invoice
STOP
```

Implementation and testing

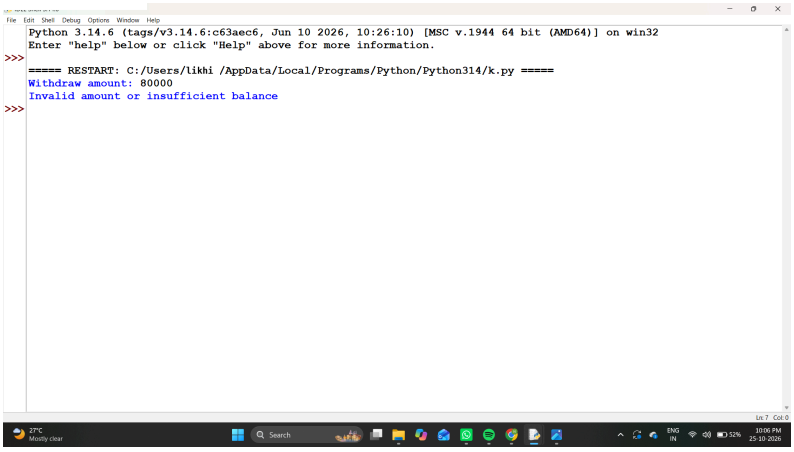
Python source code — exactly four lines

```
items=[]; n=int(input("Products: "))
for _ in range(n): name=input("Name: "); cat=input("Category grocery/clothing/electronics: ").lower(); p=float(input("Price:
sub=sum(x[1] for x in items); disc=min(sub*.1,1000) if sub>=5000 else 0; tax=sum(x[1]*(sub-disc)/sub*x[2] for x in items) if
print("Subtotal",sub,"Discount",disc,"Tax",round(tax,2),"TOTAL",round(sub-disc+tax,2))
```

Test cases

Input / scenario	Expected result
Grocery ■100 × 10	Subtotal ■1000; no discount; tax ■50; total ■1050.
Electronics ■3000 × 2	Subtotal ■6000; discount ■600; tax ■972; total ■6372.
Subtotal ■20000	Discount capped at ■1000.
Unknown category	Invalid category; item skipped.

Sample output screen



Conclusion

The invoice program combines itemized line totals, category tax rates, threshold discounts, and a maximum concession.

8. Manufacturing Cost and Profitability Analysis

Problem statement

Calculate manufacturing cost including material, labour, wastage, and machine expenses; determine unit cost, selling revenue, profit, and whether a target margin is achieved.

Objectives

• Calculate wastage-adjusted material cost.
• Combine fixed and variable production costs.
• Measure profit and compare profit margin with a target.

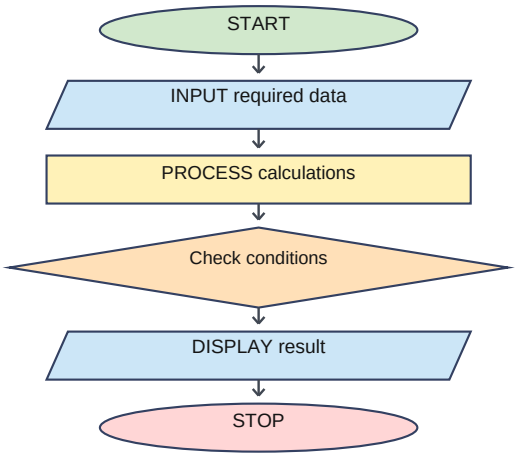
Assumptions

• Material quantity is entered in units and material rate per unit.
• Wastage is a percentage of base material cost; labour is hours × hourly rate; machine cost is entered as a total.
• Profit margin = profit / revenue × 100; production meets target if margin >= target percentage.
• All costs, quantity, and selling price must be non-negative; quantity must be greater than zero.

Algorithm

1. Read quantity, material quantity per product, material rate, wastage percentage, labour hours/rate, machine cost, selling price, and target margin.
2. Calculate base material cost and wastage cost.
3. Calculate total production cost and cost per product.
4. Calculate revenue, profit, and profit margin.
5. Compare margin with target and print the result.

Flowchart



Pseudocode

```
START
INPUT quantity, material_per_unit, material_rate, wastage_percent
INPUT labour_hours, labour_rate, machine_cost, selling_price, target_margin
material = quantity * material_per_unit * material_rate
wastage = material * wastage_percent / 100
labour = labour_hours * labour_rate
total_cost = material + wastage + labour + machine_cost
revenue = quantity * selling_price
profit = revenue - total_cost
IF revenue > 0: margin = profit / revenue * 100
ELSE margin = 0
IF margin >= target_margin: DISPLAY TARGET MET
ELSE DISPLAY TARGET NOT MET
STOP
```

Implementation and testing

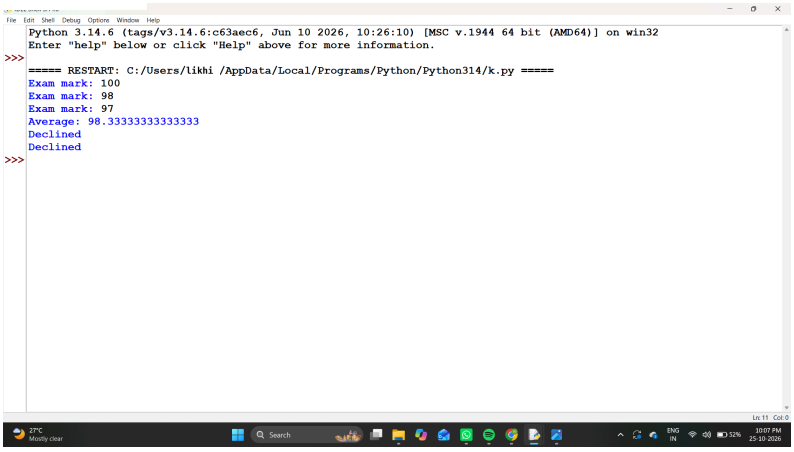
Python source code — exactly four lines

```
q=int(input("Quantity: ")); mu=float(input("Material units/product: ")); mr=float(input("Material rate: ")); w=float(input("W
lh=float(input("Labour hours: ")); lr=float(input("Labour rate: ")); machine=float(input("Machine cost: ")); price=float(inpu
mat=q*mu*mr; cost=mat+mat*w/100+lh*lr+machine; rev=q*price; profit=rev-cost; margin=profit/rev*100 if rev else 0
print("Invalid input") if q<=0 or min(mu,mr,w,lh,lr,machine,price,target)<0 else print("Cost",cost,"Revenue",rev,"Profit",pro
```

Test cases

Input / scenario	Expected result
100 products; material 2 × ■50; wastage 5%; labour ■2000; Material ■10000; wastage ■150; ■500; labour ■2000; total cost ■13500; revenue ■15000; profit ■1	
Target margin 15% with margin 10%	TARGET NOT MET.
Quantity 0	Invalid input.
Selling price 0	Margin is 0%; target not met unless target is 0.

Sample output screen



Conclusion

The cost model shows how wastage and operating costs affect profit and whether a target margin is achieved.

9. Multi-Currency Travel Budget Analysis

Problem statement

Convert travel expenses in several currencies using supplied exchange rates, include conversion fees, compare the total with a budget, and identify the largest expense category.

Objectives

• Convert foreign-currency costs into a common home currency.
• Include a conversion fee per transaction.
• Compare planned spending with a budget and find the largest category.

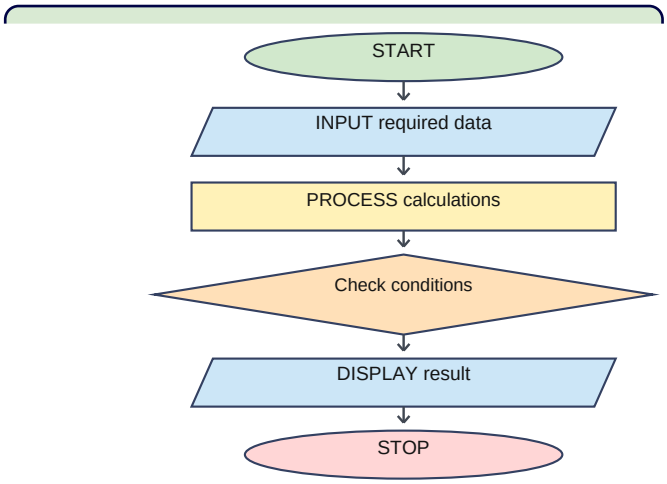
Assumptions

• Rates are home-currency units per one foreign-currency unit and are provided by the user.
• Conversion fee is 2% of converted value.
• Categories are stored as tuples (category, currency, amount, rate).
• All amounts and rates must be non-negative; a rate must be greater than zero.

Algorithm

1. Read budget and expense count.
2. For each expense, read category, currency code, amount, and exchange rate.
3. Calculate converted amount and 2% fee; add both to total and category totals.
4. Track the largest category total.
5. Compare total with budget and display over/under-budget amount.

Flowchart



Pseudocode

```
START
INPUT budget, expense_count
total = 0; category_totals = empty list
FOR each expense
    INPUT category, currency, amount, rate
    converted = amount * rate
    fee = converted * 0.02
    cost = converted + fee
    add cost to total and category total
    DISPLAY converted, fee, cost
largest = category with maximum total
IF total > budget: DISPLAY OVER BUDGET
ELSE DISPLAY WITHIN BUDGET
DISPLAY largest category
STOP
```

Implementation and testing

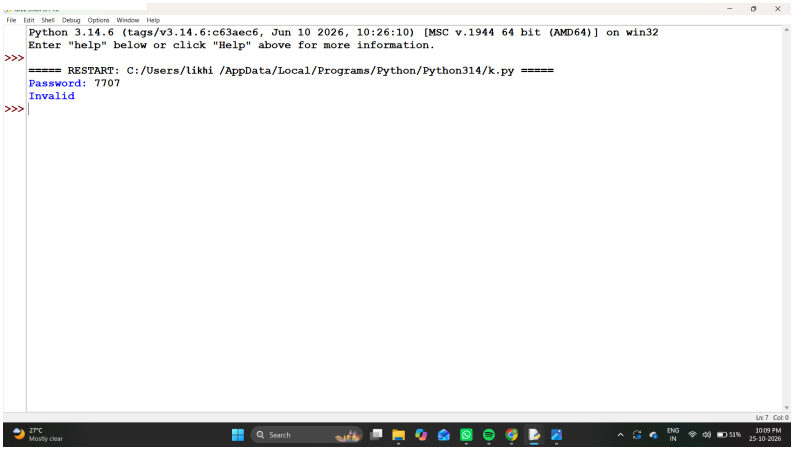
Python source code — exactly four lines

```
budget=float(input("Budget: ")); n=int(input("Expenses: ")); expenses=[]
for _ in range(n):
 cat=input("Category: "); cur=input("Currency: "); amount=float(input("Amount: ")); rate=float(input("Home currency per unit:
total=sum(x[1] for x in expenses); cats=[(cat,sum(x[1] for x in expenses if x[0]==cat)) for cat,cost in expenses]; print("Tot
```

Test cases

Input / scenario	Expected result
Hotel 100 USD at rate 83	Converted ■8300; fee ■166; total ■8466.
Budget ■10000; total ■8466	Within budget; remaining ■1534.
Transport ■5000 and hotel ■8466	Hotel is largest category.
Rate 0	Invalid exchange rate.

Sample output screen



Conclusion

The travel budget tool combines exchange-rate conversion, transaction fees, budget comparison, and largest-category analysis.

10. Campus Event Financial Planning

Problem statement

Estimate event expenses for venue, equipment, catering, transport, and certificates; calculate participant contributions, sponsorship shortfall, and minimum registrations needed to break even.

Objectives

• Sum fixed and per-participant costs.
• Calculate the funding gap after sponsorship and registration income.
• Find the minimum participant count needed to cover total costs.

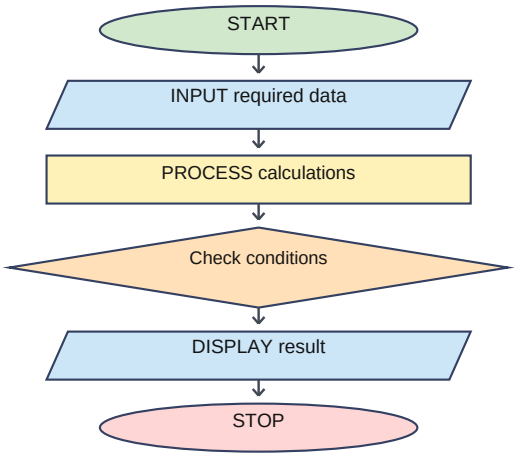
Assumptions

• Venue and equipment are fixed costs; catering, transport, and certificates are per participant.
• Registration fee is entered by the user; sponsorship is a fixed amount.
• Break-even registrations = ceiling((fixed costs - sponsorship) / (registration fee - variable cost per participant)) when fee exceeds variable cost.
• If registration fee is less than or equal to variable cost per participant, registration alone cannot break even.

Algorithm

1. Read fixed costs, per-participant costs, expected participants, registration fee, and sponsorship.
2. Calculate fixed cost, variable cost per participant, total event cost, registration income, and funding gap.
3. Calculate net contribution per participant.
4. If net contribution is positive, calculate minimum registrations using a loop until income covers fixed cost after sponsorship.
5. Display budget and whether planned registrations break even.

Flowchart



Pseudocode

```
START
INPUT venue, equipment, catering_per_person, transport_per_person
INPUT certificate_per_person, expected_people, fee, sponsorship
fixed = venue + equipment
variable = catering_per_person + transport_per_person + certificate_per_person
total_cost = fixed + variable * expected_people
income = fee * expected_people + sponsorship
gap = total_cost - income
IF fee > variable
    required = 0
    WHILE required*fee + sponsorship < fixed + required*variable
        required = required + 1
    DISPLAY required
ELSE DISPLAY BREAK-EVEN IMPOSSIBLE FROM REGISTRATION ALONE
DISPLAY costs, income, gap
STOP
```

Implementation and testing

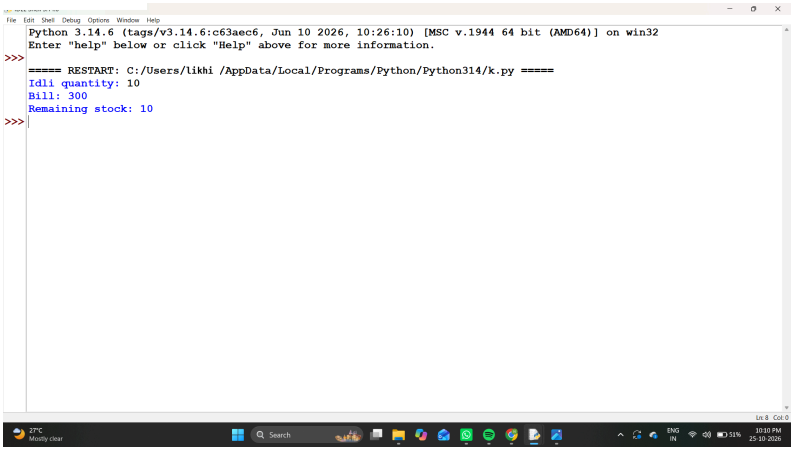
Python source code — exactly four lines

```
v=float(input("Venue: ")); e=float(input("Equipment: ")); c=float(input("Catering/person: ")); t=float(input("Transport/perso
p=int(input("Participants: ")); fee=float(input("Registration fee: ")); s=float(input("Sponsorship: ")); fixed=v+e; var=c+t+c
print("Invalid input") if min(v,e,c,t,cert,fee,s)<0 or p<0 else print("Break-even impossible") if fee<=var else print("Minimu
# Break-even uses the ceiling of fixed cost after sponsorship divided by net contribution.
```

Test cases

Input / scenario	Expected result
Fixed ■20000; variable ■300/person; fee ■500; sponsorship ■5000	Break-even contribution ■200/person; required registrations = 75.
Expected 50 people; same values	Cost ■35000; income ■30000; shortfall ■5000.
Fee ■250, variable ■300	Registration alone cannot break even.
Negative venue cost	Invalid input.

Sample output screen



Conclusion

The event planner estimates the funding gap and calculates minimum registrations needed to break even.

Overall conclusion

These ten case studies demonstrate introductory Python problem-solving: identify inputs and rules, decompose calculations, write algorithms and pseudocode, map control flow, implement code, and test normal and boundary cases. The four-line examples are intentionally compact for the requested format; expand them and strengthen validation before real-world use.